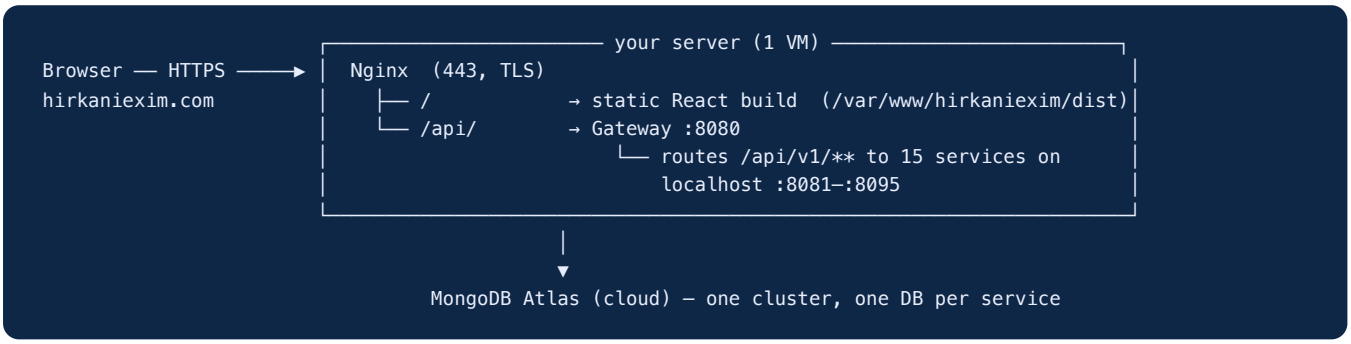
HIRKANI EXIM

Deploying Hirkani Exim to hirkaniexim.com

A start-to-finish guide to run the whole platform on one server, behind your domain, over HTTPS. Everything (marketing site, company app, admin portal, all APIs) is reached through `https://hirkaniexim.com`.

1. How it fits together (read this first)



- **One origin.** The React app calls `/api/v1/...` (relative). Because Nginx serves the

site *and* proxies `/api` on the same domain, there is **no CORS to configure**.

- **One gateway.** Only `:8080` is reached by Nginx; it fans out to the 15 services on

localhost. Those service ports are never exposed publicly.

- **16 JVMs.** This is the big cost: 15 services + the gateway. Plan RAM accordingly

(see §2). MongoDB itself is managed by Atlas, not on your VM.

Service	Port		Service	Port
gateway	8080		logistics	8089
identity	8081		payments	8090
verification	8082		billing	8091
catalog	8083		trust	8092
sourcing	8084		notification	8093
quotation	8085		admin	8094
messaging	8086		activity	8095
orders	8087			
documents	8088			

2. What you need

1. **A server (VM)** with a public IP. Ubuntu 22.04 LTS.

- **RAM: 8 GB minimum, 16 GB recommended.** 16 Java services at `-Xmx256m` fit in ~6–7 GB;

16 GB gives breathing room. 2 GB will *not* work.

- 2–4 vCPU, 30+ GB disk.
- Providers: AWS EC2 (`t3.large` / `t3.xlarge`), DigitalOcean/Hetzner (8–16 GB droplet), GCP, Azure.

1. **The domain** `hirkaniexim.com` (you have it) — access to its DNS records.
2. **MongoDB Atlas** — your existing cluster + the connection string.
3. **Secrets**: a `JWT_SECRET`, an `ADMIN_BOOTSTRAP_SECRET`, an admin password (generated in §6).

---

3. Point the domain at the server (DNS)

You need two records pointing at your server's public IP:

Type	Name (Host)	Value	TTL
A	`@`	`YOUR.SERVER.IP`	600
CNAME	`www`	`hirkaniexim.com`	1 hour

(An `A www → YOUR.SERVER.IP` record works too — either is fine.)

Doing it in GoDaddy (your registrar)

The registrar doesn't change anything in this project — only *where you click* to set DNS.

1. Sign in to GoDaddy → top-right menu → **Domain Portfolio** (or "My Products").
2. Find `hirkaniexim.com` → the ⋮ **/ DNS** button → **Manage DNS**.
3. GoDaddy pre-creates a **parked** `A` record on `@` and a `CNAME www → @`. **Edit the

existing `A @` record — don't add a second one: **set its** Value **to** `YOUR.SERVER.IP`**, Save. Optionally lower** TTL** to 600 seconds for faster propagation.

1. Leave (or recreate) the `CNAME` `www → @` so `www.hirkaniexim.com` follows the apex.
2. **Do NOT use GoDaddy "Domain Forwarding"** and don't point the domain at a GoDaddy

"Website Builder"/parking product — those override your A record. Plain DNS A/CNAME only.

1. Keep GoDaddy's **default nameservers** (managing DNS here is simplest). Only change

nameservers if you deliberately move DNS to another provider (e.g. Cloudflare).

1. MX/email records are unaffected — leave them as-is.

DNS can take 5 minutes to a few hours to propagate. Verify from your laptop:

```
dig +short hirkaniexim.com        # should print YOUR.SERVER.IP
dig +short www.hirkaniexim.com    # should resolve to the same IP
```

Do not continue to TLS (§9) until both resolve — Let's Encrypt validates over the live domain.

---

4. Allow the server in MongoDB Atlas

Atlas blocks unknown IPs by default.

1. Atlas → **Network Access** → **Add IP Address** → enter your server's public IP (`/32`).

(Quick demo only: `0.0.0.0/0` allows anywhere — fine to test, tighten later.)

1. Atlas → **Database Access** → confirm a DB user + password; that's what goes in `MONGODB_URI`.
2. Each service writes to its **own** database (`exim_identity`, `exim_catalog`, …) on the one

cluster — already configured; nothing to create, Mongo makes them on first write.

---

5. Prepare the server (one-time)

SSH in as a sudo user, then:

```
# Java 17 (the app targets 17) + Nginx + tools
sudo apt update
sudo apt install -y openjdk-17-jre-headless nginx ufw

# A dedicated, non-login user to run the services
sudo useradd --system --create-home --shell /usr/sbin/nologin exim
sudo mkdir -p /opt/exim/jars /var/www/hirkaniexim

# Firewall: only SSH + web open to the world (service ports stay private)
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw --force enable
```

6. Build the artifacts (on your machine)

From the repo root:

```
# 1) Backend: build all 16 fat-jars
mvn clean package -DskipTests

# 2) Collect them into deploy/jars/ with clean names (identity-service.jar, gateway.jar, …)
./deploy/collect-jars.sh

# 3) Frontend: build the static site
cd frontend && npm ci && npm run build && cd ..
#   -> produces frontend/dist/
```

Generate your secrets now (keep them safe):

```
openssl rand -base64 48   # use for JWT_SECRET
openssl rand -base64 24   # use for ADMIN_BOOTSTRAP_SECRET
```

7. Upload everything to the server

```
# Backend jars  ->  /opt/exim/jars/
scp deploy/jars/*.jar         youruser@YOUR.SERVER.IP:/tmp/jars/
ssh youruser@YOUR.SERVER.IP 'sudo mv /tmp/jars/*.jar /opt/exim/jars/ && sudo chown -R exim:exim /opt/exim'

# Frontend build -> /var/www/hirkaniexim/dist
scp -r frontend/dist          youruser@YOUR.SERVER.IP:/tmp/dist/
ssh youruser@YOUR.SERVER.IP 'sudo rm -rf /var/www/hirkaniexim/dist && sudo mv /tmp/dist
/var/www/hirkaniexim/dist'

# Env + systemd template + nginx config
scp deploy/exim.env.example         youruser@YOUR.SERVER.IP:/tmp/exim.env
scp deploy/systemd/exim@.service    youruser@YOUR.SERVER.IP:/tmp/
scp deploy/nginx/hirkaniexim.com.conf youruser@YOUR.SERVER.IP:/tmp/
```

On the server, place the env file and fill in real values:

```
sudo mv /tmp/exim.env /opt/exim/exim.env
sudo nano /opt/exim/exim.env          # paste MONGODB_URI, JWT_SECRET, ADMIN_BOOTSTRAP_SECRET, admin password
sudo chown exim:exim /opt/exim/exim.env && sudo chmod 600 /opt/exim/exim.env
```

8. Run the backend services (systemd)

```
sudo mv /tmp/exim@.service /etc/systemd/system/
sudo systemctl daemon-reload

# Start all 16 (gateway last). Each jar knows its own port from its application.yml.
for s in identity-service verification-service catalog-service sourcing-service \
         quotation-service messaging-service orders-service documents-service \
         logistics-service payments-service billing-service trust-service \
         notification-service admin-service activity-service gateway; do
  sudo systemctl enable --now exim@$s
  sleep 3
done
```

Check them (they need ~20–40s to connect to Atlas):

```
systemctl list-units 'exim@*'                    # all should be "active (running)"
curl -s localhost:8080/actuator/health           # gateway -> {"status":"UP"}
journalctl -u exim@identity-service -n 50 --no-pager # logs for one service
```

> Tip: start services a few seconds apart (the loop does this) so 16 JVMs don't hammer Atlas and CPU all at once on a small box.

9. Web server + HTTPS

```
# Site config
sudo mv /tmp/hirkaniexim.com.conf /etc/nginx/sites-available/hirkaniexim.com
sudo ln -s /etc/nginx/sites-available/hirkaniexim.com /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx

# Free TLS certificate from Let's Encrypt (auto-renews). DNS from §3 must resolve first.
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d hirkaniexim.com -d www.hirkaniexim.com
#   choose "redirect HTTP -> HTTPS" when asked.
```

Certbot rewrites the Nginx file to listen on 443 with your certificate and redirect port 80.

10. Verify the whole thing through the domain

```
curl -s https://hirkaniexim.com/api/v1/plans           # public-ish API via gateway (JSON)
curl -s https://hirkaniexim.com/ | head                 # the SPA's index.html
```

Then in a browser:

- **Marketing site** → `https://hirkaniexim.com/`
- **Company app** → `https://hirkaniexim.com/app` (register a company, log in)
- **Admin portal** → `https://hirkaniexim.com/admin/login`

(sign in with `PLATFORM_ADMIN_EMAIL` / the password you set — the seeder created it on first boot).

If all three load and you can register → log in → see data, you're live. 🎉

11. Redeploying after code changes

```
# Backend changed:
mvn clean package -DskipTests && ./deploy/collect-jars.sh
scp deploy/jars/*.jar youruser@SERVER:/tmp/jars/
ssh youruser@SERVER 'sudo mv /tmp/jars/*.jar /opt/exim/jars/ && sudo chown exim:exim /opt/exim/jars/*.jar &&
sudo systemctl restart "exim@*"'

# Frontend changed:
cd frontend && npm run build && cd ..
scp -r frontend/dist youruser@SERVER:/tmp/dist/
ssh youruser@SERVER 'sudo rm -rf /var/www/hirkaniexim/dist && sudo mv /tmp/dist /var/www/hirkaniexim/dist'
# (no restart needed for frontend; hard-refresh the browser)
```

12. Troubleshooting

Symptom	Likely cause / fix
`502 Bad Gateway` on `/api/...`	Gateway not up. `systemctl status exim@gateway`, `journalctl -u exim@gateway`.
API returns `... status: DOWN` but works	Atlas free-tier health-probe quirk — harmless; the app still serves.
Services crash on boot, logs show Mongo timeout	Server IP not allowed in Atlas (§4), or wrong `MONGODB_URI`.
`/app` or `/admin` 404 on refresh	Nginx SPA fallback missing — ensure `try_files $uri /index.html` (it's in the provided conf).
Login works then 403 after 15 min	Expected — JWT lifetime is 15 min, no refresh token yet. Log in again.
Out-of-memory / box swapping	Too little RAM for 16 JVMs. Lower `-Xmx` or size up to 16 GB.
Admin login rejects your account	That account isn't PLATFORM_ADMIN/SUPPORT. Use the seeded admin, or `POST /api/v1/auth/bootstrap-admin` with `ADMIN_BOOTSTRAP_SECRET`.

13. Production hardening (after it's working)

These are deliberately **not** done yet (see CLAUDE.md "deferred"):

- **Change default secrets** — `JWT_SECRET`, `ADMIN_BOOTSTRAP_SECRET`, the admin password.
- Real file upload for KYC (today `fileUrl` is a string — no object storage).
- Refresh tokens; per-company ownership checks; rate limiting; observability/metrics.
- Per-service container images + an orchestrator if you outgrow one VM.
- Backups: Atlas has automated backups — confirm they're enabled on your tier.

> **Scaling note:** one VM hosting 16 JVMs is fine for a demo/MVP. For real traffic, containerize each service and move to a managed platform (ECS/Fargate, GKE, Kubernetes), putting the gateway behind a load balancer. The app is already structured for that (each service is independently deployable and owns its own database).